

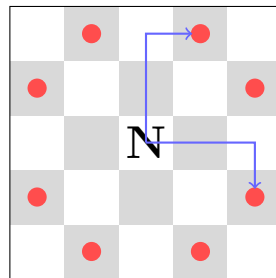
The Knight's Tour

1 The knight and its graph

A knight on square (x, y) may jump to any of the eight squares

$$(x \pm 1, y \pm 2), \quad (x \pm 2, y \pm 1) \quad (1)$$

that lie on the board. Abstracting the chessboard away, the $N \times N$ *knight graph* has one vertex per square and one edge per legal jump; everything below is graph theory in chess clothing.



The eight knight moves; two are drawn as their L shapes.

2 Open and closed tours

A *tour* visits every square of the board exactly once.

- An **open tour** may end anywhere: a *Hamiltonian path* of the knight graph.
- A **closed tour** (or re-entrant tour) ends a single knight's move away from its starting square, so the knight could go round forever: a *Hamiltonian cycle*.

Existence is completely settled on square boards:

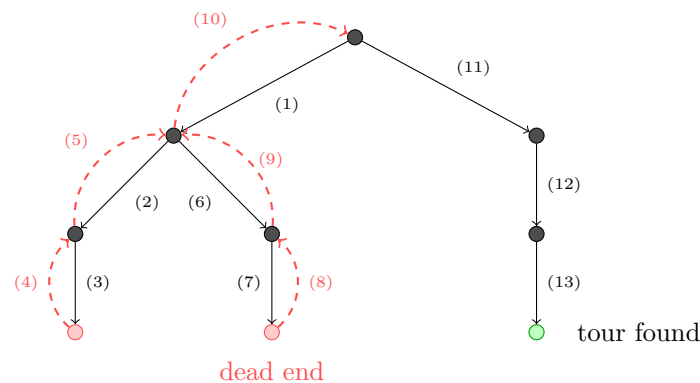
- Open tours exist exactly for $N = 1$ and $N \geq 5$. There is no tour on 2×2 or 3×3 (on the latter, the center square has no knight move at all), and — less obviously — none on 4×4 either.
- Closed tours exist exactly for even $N \geq 6$.

The parity half of the closed-tour statement has a one-line proof worth savoring. Every knight move changes square color, so any walk alternates black and white. A closed tour is a cycle through all N^2 squares using each exactly once; alternation around a cycle forces equally many squares of each color, hence N^2 even, hence N even. For odd N the board has one color in excess and no closed tour can exist — an impossibility established without examining a single candidate tour, just from an invariant every move preserves.

3 Algorithms

3.1 Backtracking

The direct search extends a partial path square by square: from the current square, try each unvisited neighbor in turn, recurse, and undo on failure; success is a path of length N^2 .

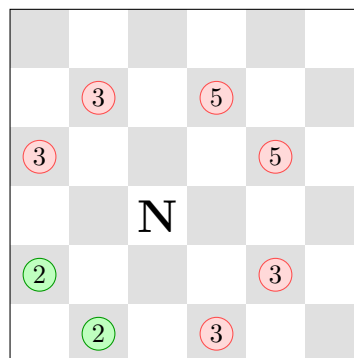


The search tree, edges numbered in traversal order: solid arrows descend, dashed red arrows undo after a dead end.

This always works in principle — it is exhaustive — but the knight graph is generous (up to eight choices per step, path length N^2), and from an unlucky start the tree above can grow astronomically many red leaves before its first green one. Unlike the queens problem, where each placement prunes entire columns and diagonals, a knight move constrains the future only weakly, so whole subtrees are explored in vain.

3.2 Warnsdorff's rule

The classical remedy (1823) is a greedy ordering: *always jump to the unvisited square that itself has the fewest unvisited onward moves*. Here is the rule at work on an empty 6×6 board, every candidate labeled with its count of onward moves:



Warnsdorff picks a green square: only two ways onward from there.

The intuition is a rescue principle: the squares near the corner have the fewest ways in and out, so they are the ones in danger of becoming unreachable islands — visit them while it is still possible, and leave the well-connected center for later. With this ordering the very first descent of the search tree almost always runs straight to a green leaf: backtracking in name, linear in practice.

Two honest caveats, one of them visible above: the two green squares *tie* at count 2, and ties must be broken somehow — on larger boards a poor tie-breaking rule can strand the pure heuristic in a dead end. Keeping the backtracking machinery underneath restores certainty at

no cost when the heuristic behaves. And greedy success is empirical, not guaranteed: the rule is a heuristic, not a theorem.

4 From mathematics to program

The program searches for *closed* tours. It represents the board as a visited matrix plus the trail of moves so far; the eight jump offsets are a constant list. A candidate is accepted when the path covers all N^2 squares *and* its last square is a knight's move from the first — the extra closure test is the only difference from an open-tour search. The result is an *option*: **Some tour** on first success — there is no point enumerating further — or **None**, which for odd N or $N < 6$ is not a failure of the search but a fact of mathematics. Sorting candidate moves by onward degree turns the naive search into Warnsdorff's; timing both against each other makes the difference measurable rather than anecdotal.